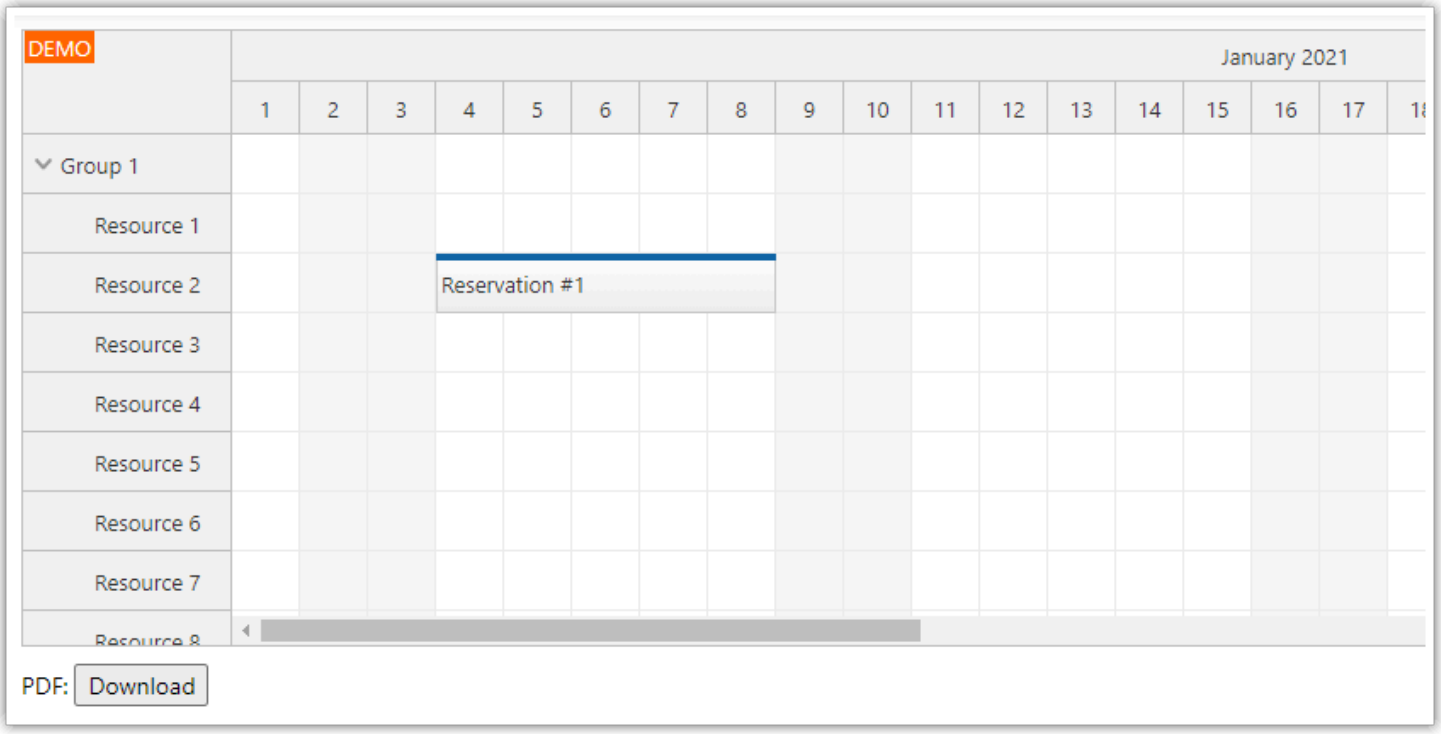
JavaScript/HTML5 Scheduler PDF Export (Paged by Month)

Simple HTML web application that exports the Scheduler to PDF (client-side).

Tags: pdf javascript export jpeg html5 scheduler



Downloads	Download Source	JavaScript project with source code for download. *328 kB*
Languages	JavaScript, HTML5	
Technologies	PDF, JPEG	

Features

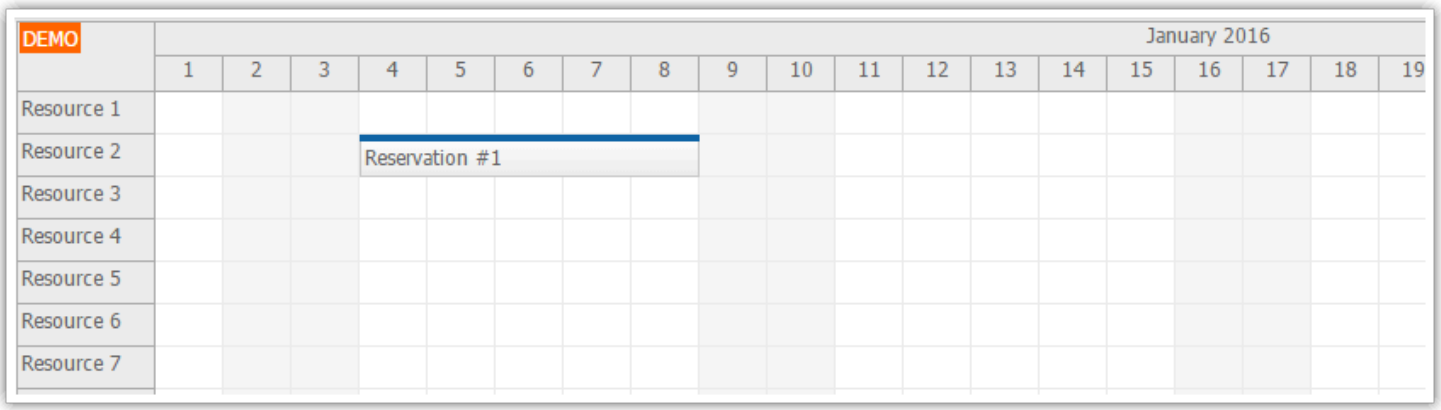
- Client-side PDF export of HTML5 Scheduler (pure JavaScript)
- Uses jsPDF library (open-source, MIT license)
- Uses DayPilot Pro for JavaScript (trial version)
- Generates multi-page PDF, one month per page

To learn how to export the React Scheduler component to PDF, please see the React Scheduler: PDF Export/Printing tutorial.

License

Licensed for testing and evaluation purposes. Please see the license agreement included in the sample project. You can use the source code of the tutorial if you are a licensed user of DayPilot Pro for JavaScript.

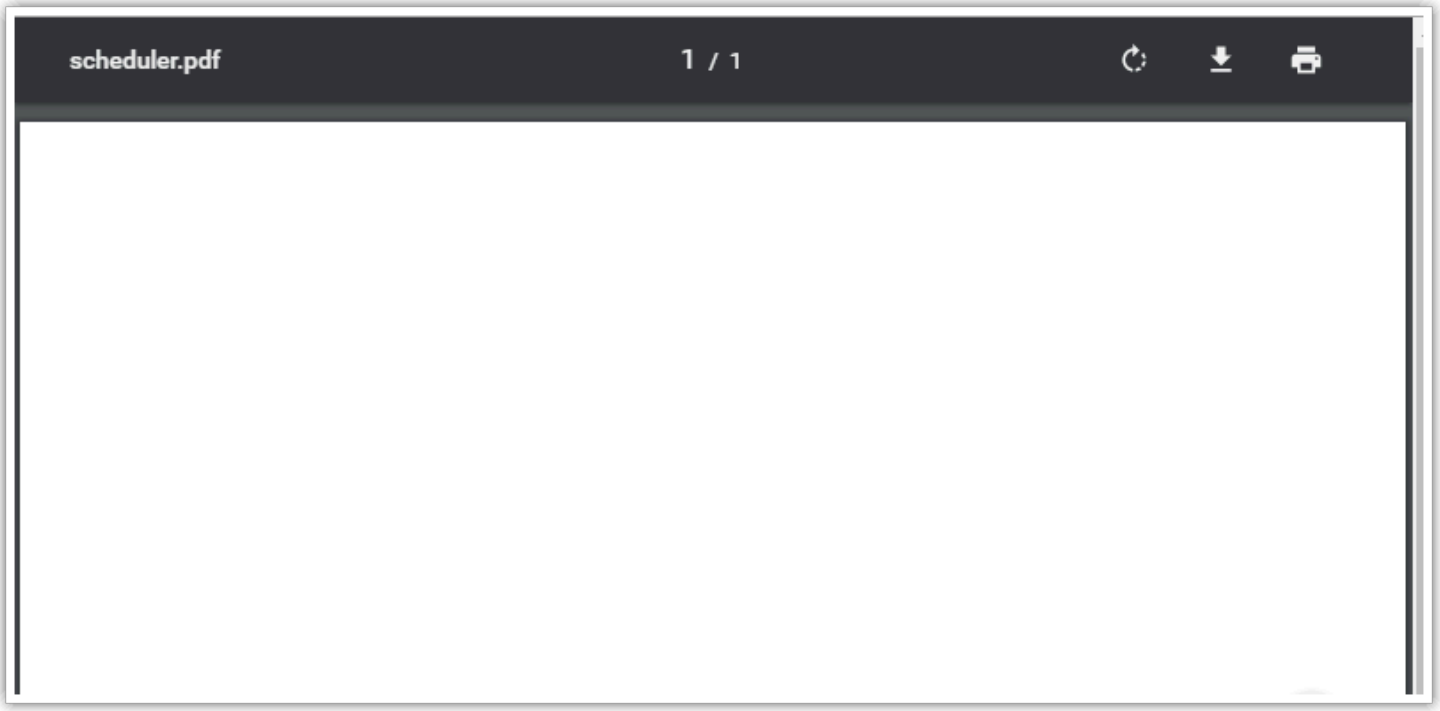
Initialize the HTML5 Scheduler



We will create a DayPilot JavaScript Scheduler instance that will display a timeline (one day per cell) for 20 resources:

```
<div id="dp"></div>

<script>
    var dp = new DayPilot.Scheduler("dp");
    dp.scale = "Day";
    dp.timeHeaders = [
        { groupBy: "Month"},
        { groupBy: "Day", format: "d"}
    ];
    dp.heightSpec = "Max";
    dp.height = 300;

    dp.startDate = new DayPilot.Date("2021-01-01");
    dp.days = 366;
    dp.resources = [
        {name: "Resource 1", id: 1},
        {name: "Resource 2", id: 2},
        // ...
    ];
    dp.events.list = [
        {
            start: "2021-01-04",
            end: "2021-01-09",
            id: 1,
            resource: 2,
            text: "Reservation #1"
        }

    ];
    dp.init();

</script>
```

Create a New PDF Document



We will use the open-source jsPDF library to create the PDF document.

- Open-source (MIT license)
- About 290 kB of minified code
- API documentation: https://mrrio.github.io/jsPDF/doc/

Creating a new PDF document

```
function createPdfAsBlob() {
  var doc = new jsPDF("landscape", "in", "letter");
  return doc.output("blob");
}
```

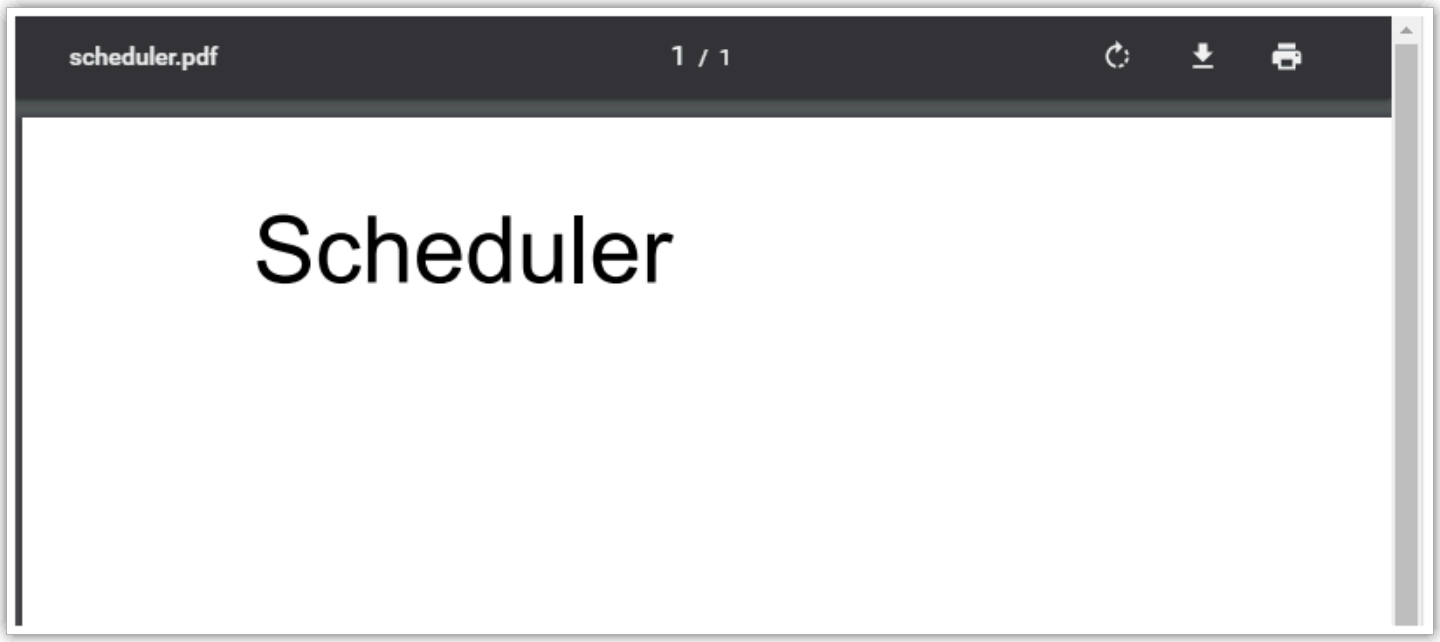
The jsPDF constructor lets you define the PDF document properties:

```
new jsPDF(orientation, unit, size);
```

Parameters:

- orientation: "landscape" | "portrait" (default)
- unit: "pt" | "cm" | "in" | "mm" (default)
- page size: "a0" - "a10" | "b0" - "b10" | "c0" - "c10" | "d1" | "letter" | "government-letter" | "legal" | "junior-legal" | "ledger" | "tabloid" | "credit-card" (default is "a4")

Add a Header to the PDF Document



We will add a header ("Scheduler" text) to the PDF document using **`jsPDF.text()`** method:

```
function createPdfAsBlob() {
  var doc = new jsPDF("landscape", "in", "letter");
  doc.setFontSize(40);
  doc.text(0.5, 1, "Scheduler");
  return doc.output("blob");
}
```

Add a JPEG Image

You can add a JPEG image to the PDF file using **`addImage(dataURI, format, x, y, width, height)`** method:

```
function createPdfAsBlob() {
  var doc = new jsPDF("landscape", "in", "letter");
  doc.setFontSize(40);
  doc.text(0.5, 1, "Scheduler");

  doc.addImage("....", 'JPEG', 1, 2, 4, 2);

  return doc.output("blob");
}
```

Add a PNG Image

jsPDF also supports PNG images:

```
function createPdfAsBlob() {
  var doc = new jsPDF("landscape", "in", "letter");
  doc.setFontSize(40);
  doc.text(0.5, 1, "Scheduler");

  doc.addImage("....", 'PNG', 1, 2, 4, 2);

  return doc.output("blob");
}
```

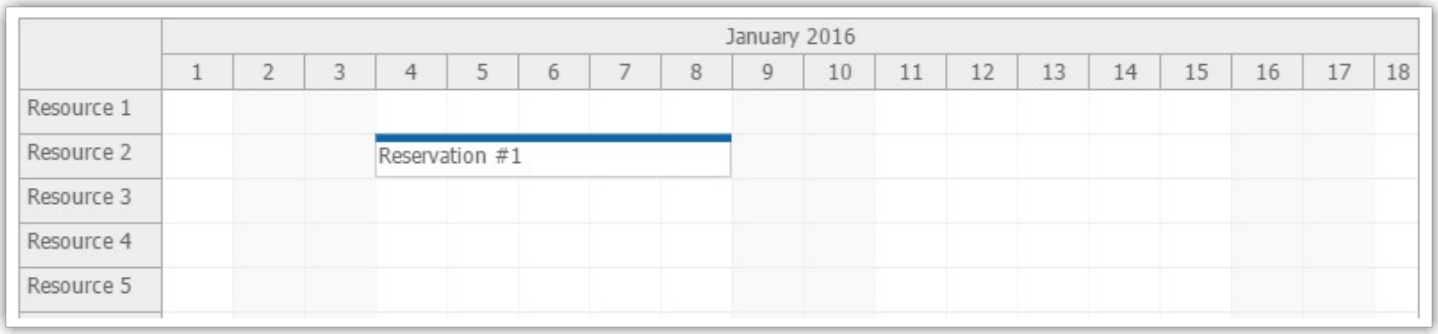
However, jsPDF seems to include the PNG filed uncompressed in the PDF file which makes the output extremely big.

The same PDF file with 12 images exported from the Scheduler (one per month):

- JPEG images: 2,89 MB
- PNG images: 128 MB

In this sample we will use JPEG format for the Scheduler export.

Export the Scheduler Viewport as a JPEG Image



This code will export the current Scheduler viewport as a JPEG image.

```
var image = dp.exportAs("jpeg").toDataUri();
```

Export the Full Scheduler

If you add `{ area: "full" }` options parameter the exported image will include the full Scheduler grid (as defined using `startDate` and `days` properties).

```
var image = dp.exportAs("jpeg", { area: "full" }).toDataUri();
```

Export a Time Range

We will use `{area: "range"}` option and export only a selected time range (January 2021):

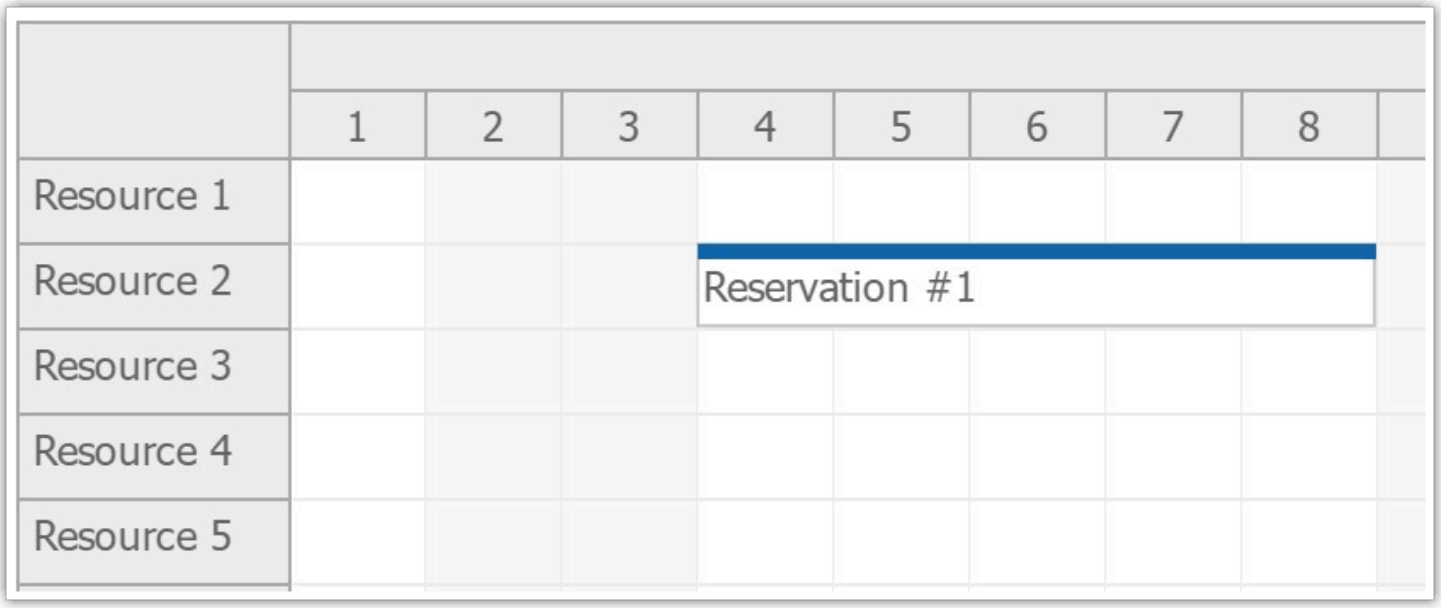
```
var image = dp.exportAs("jpeg", {
  area: "range",
  dateFrom: "2021-01-01",
  dateTo: "2021-02-01",
}).toDataUri();
```

Set the JPEG Image Quality

When exporting the Scheduler to JPEG, it is possible to specify the output JPEG quality (0-1, the higher the better quality):

```
var image = dp.exportAs("jpeg", {
  area: "range",
  dateFrom: "2021-01-01",
  dateTo: "2021-02-01",
  quality: 0.95
}).toDataUri();
```

Adjust the Image DPI



The exported image that includes January is 1320 pixels wide. If we include this image in a Letter-size landscape page (11 inches wide minus 1 inch for borders) we get a 132 DPI resolution:

```
dpi = 1320 / (11 - 1) = 132
```

Most office printers operate at 300 dpi so we need to export the image in a higher resolution.

Increasing the scale export parameter to 2 will produce a 2640px image which gets us twice the resolution (264 dpi):

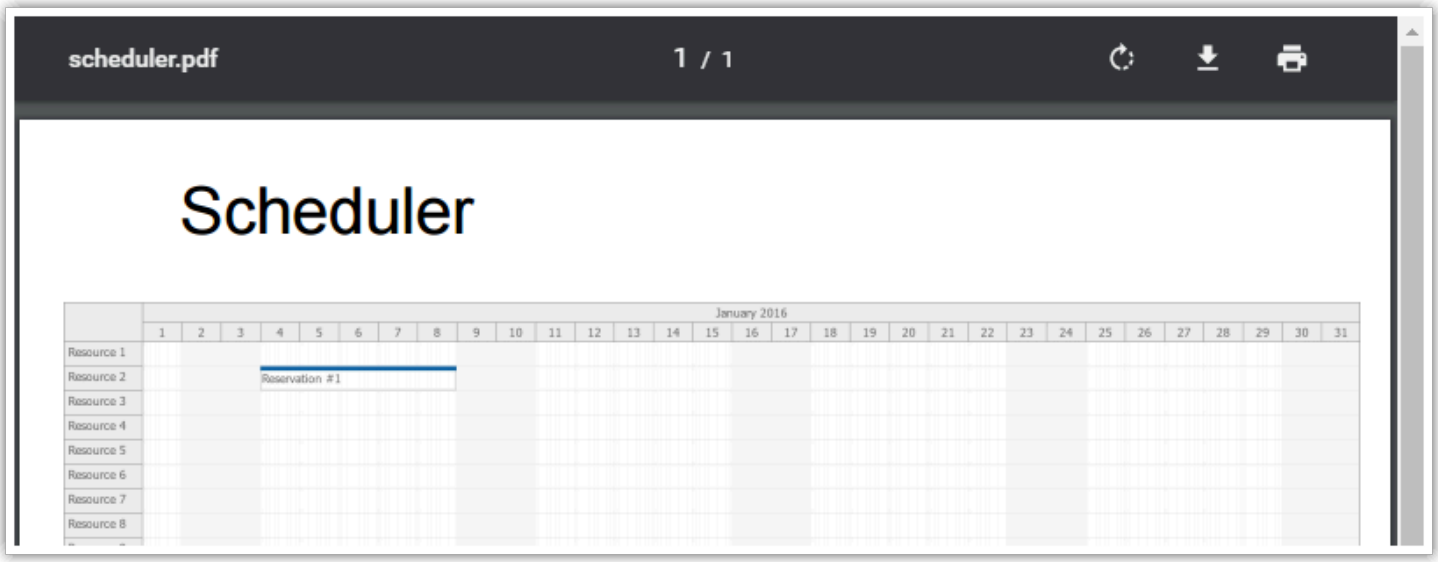
```
dpi = 2540 / (11 - 1) = 264
```

This will result in much better print quality.

Our export configuration now looks like this:

```
var image = dp.exportAs("jpeg", {
  area: "range",
  scale: 2,
  dateFrom: "2021-01-01",
  dateTo: "2021-02-01",
  quality: 0.95
}).toDataUri();
```

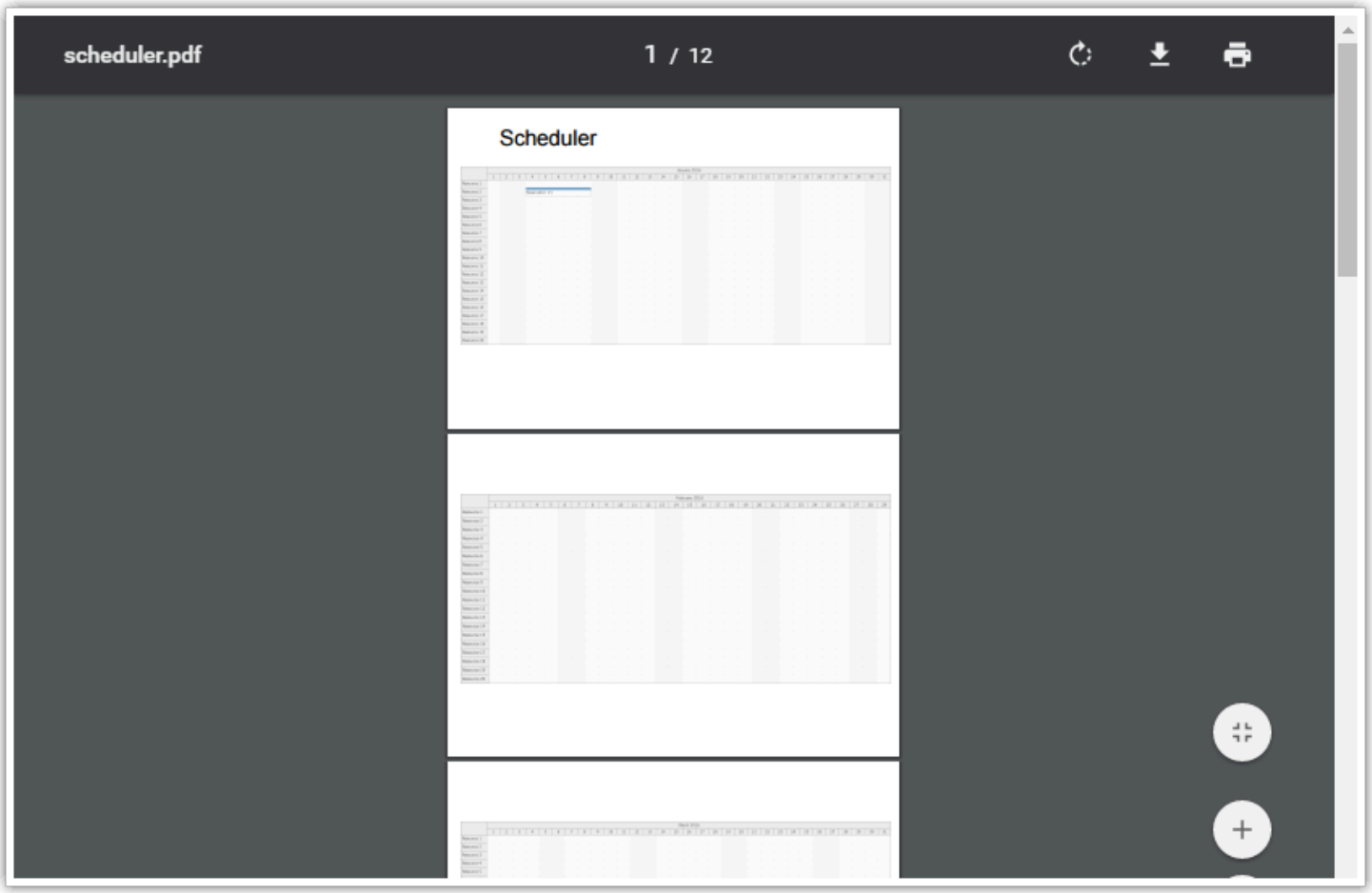
Insert the Image in the PDF Document



The following function will export January 2021 and add it to a new PDF document:

```
function createPdfAsBlob() {

  var doc = new jsPDF("landscape", "mm", "a4");
  doc.setFontSize(40);
  doc.text(35, 25, "Scheduler");

  var image = dp.exportAs("jpeg", {
        area: "range",
        scale: 2,
        dateFrom: "2021-01-01",
        dateTo: "2021-02-01",
        quality: 0.95
  });
  var dimensions = image.dimensions();
  var ratio = dimensions.width / dimensions.height;
  var width = 280;
  var height = width/ratio;
  doc.addImage(image.toDataUri(), 'JPEG', 10, 40, width, height);

  return doc.output("blob");
}
```

Export the Scheduler as a Multi-Page PDF Document (One Month per Page)



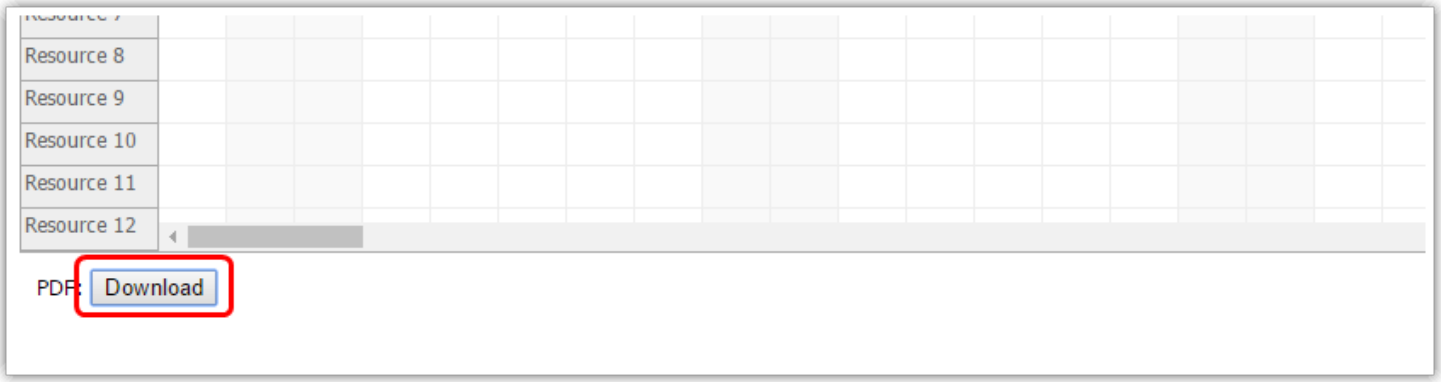
We want to export the full Scheduler grid. However, the grid is too wide and doesn't fit a single PDF page.

The updated **`createPdfAsBlob()`** function add each months to a special page:

```
function createPdfAsBlob() {
    var doc = new jsPDF("landscape", "mm", "a4");
    doc.setFontSize(40);
    doc.text(35, 25, "Scheduler");

    for (var i = 1; i <= 12; i++) {
        var image = dp.exportAs("jpeg", {
            area: "range",
            scale: 2,
            dateFrom: dp.startDate.addMonths(i-1),
            dateTo: dp.startDate.addMonths(i),
            quality: 0.95
        });
        var dimensions = image.dimensions();
        var ratio = dimensions.width / dimensions.height;
        var width = 280;
        var height = width/ratio;
        doc.addImage(image.toDataUri(), 'JPEG', 10, 40, width, height);

        var last = i === 12;
        if (!last) {
```

```
            doc.addPage();
        }
    }

    return doc.output("blob");
}
```

Download the PDF Document



Now we have the PDF document available as a `Blob` .

```
<div id="out" style="margin:10px;">PDF: <button id="download">Download</button></div>

<script>
  const elements = {
    download: document.querySelector("#download")
  };

  elements.download.addEventListener("click", () => {
    const blob = createPdfAsBlob();
    DayPilot.Util.downloadBlob(blob, "scheduler.pdf");
  });
</script>
```

Share

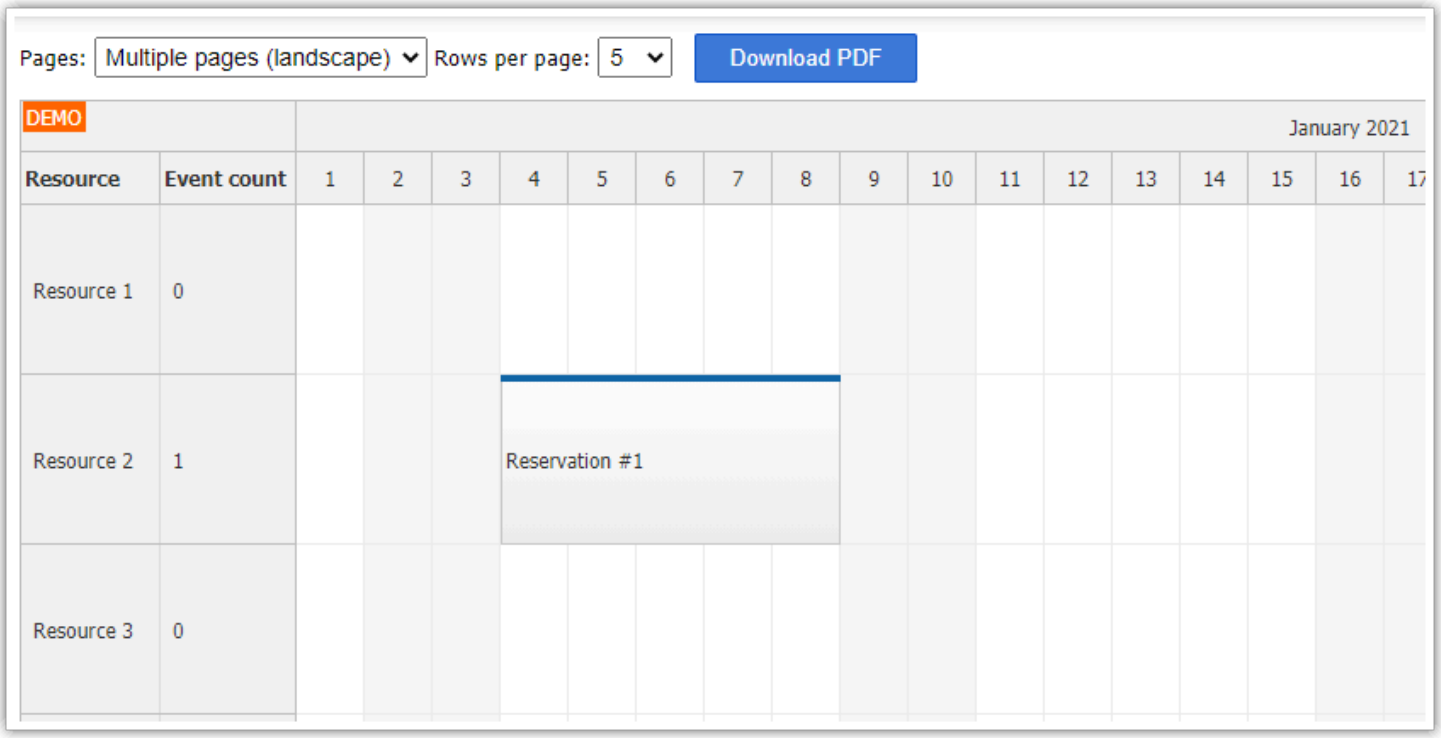
Related Questions

Problem with events and resources background (5 replies) asked by Arpine on Jul 30, 2021

Problem with exporting events (2 replies) asked by Arpine on Jul 12, 2021

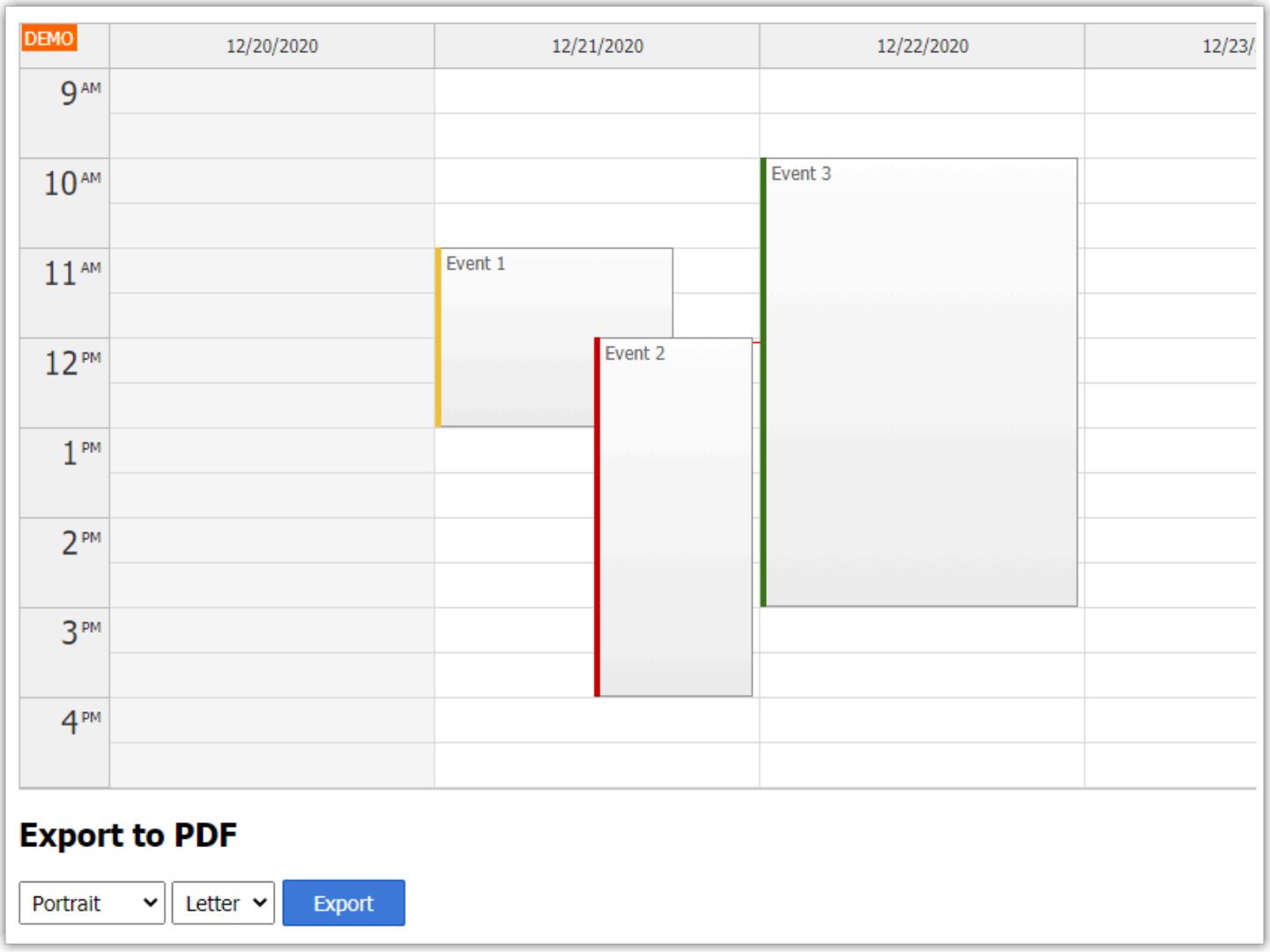
See all related questions [forums.daypilot.org]

Ask a Question



JavaScript/HTML5 Scheduler PDF Export (Paged by Resources)

HTML5 sample that shows how to export the JavaScript Scheduler to multi-page PDF file (5 rows per page).

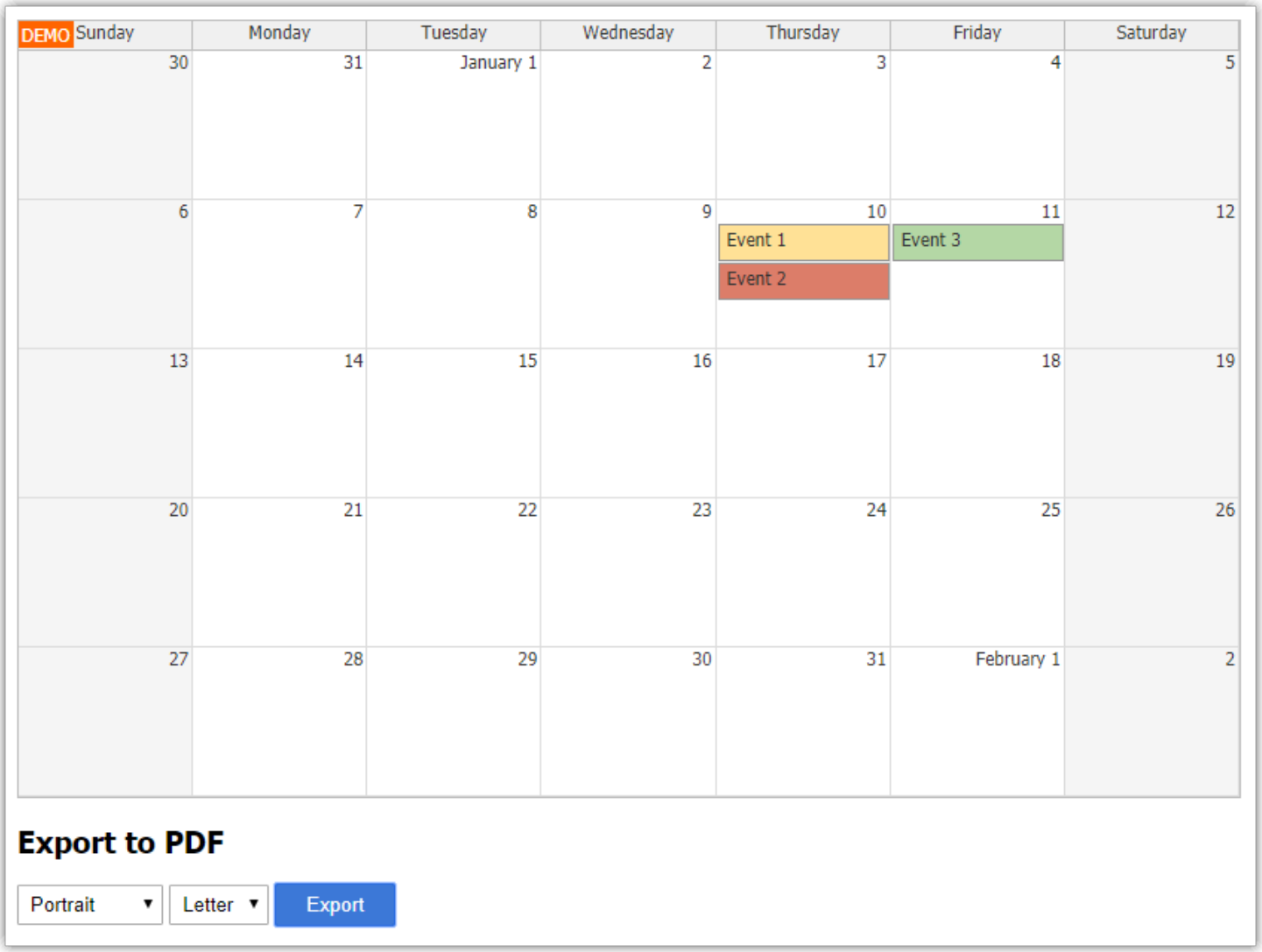


JavaScript Calendar: PDF Export Tutorial

A simple HTML5/JavaScript web application that exports the current view of the JavaScript Calendar component (including appointments) as a PDF document. Pure client-side solution.

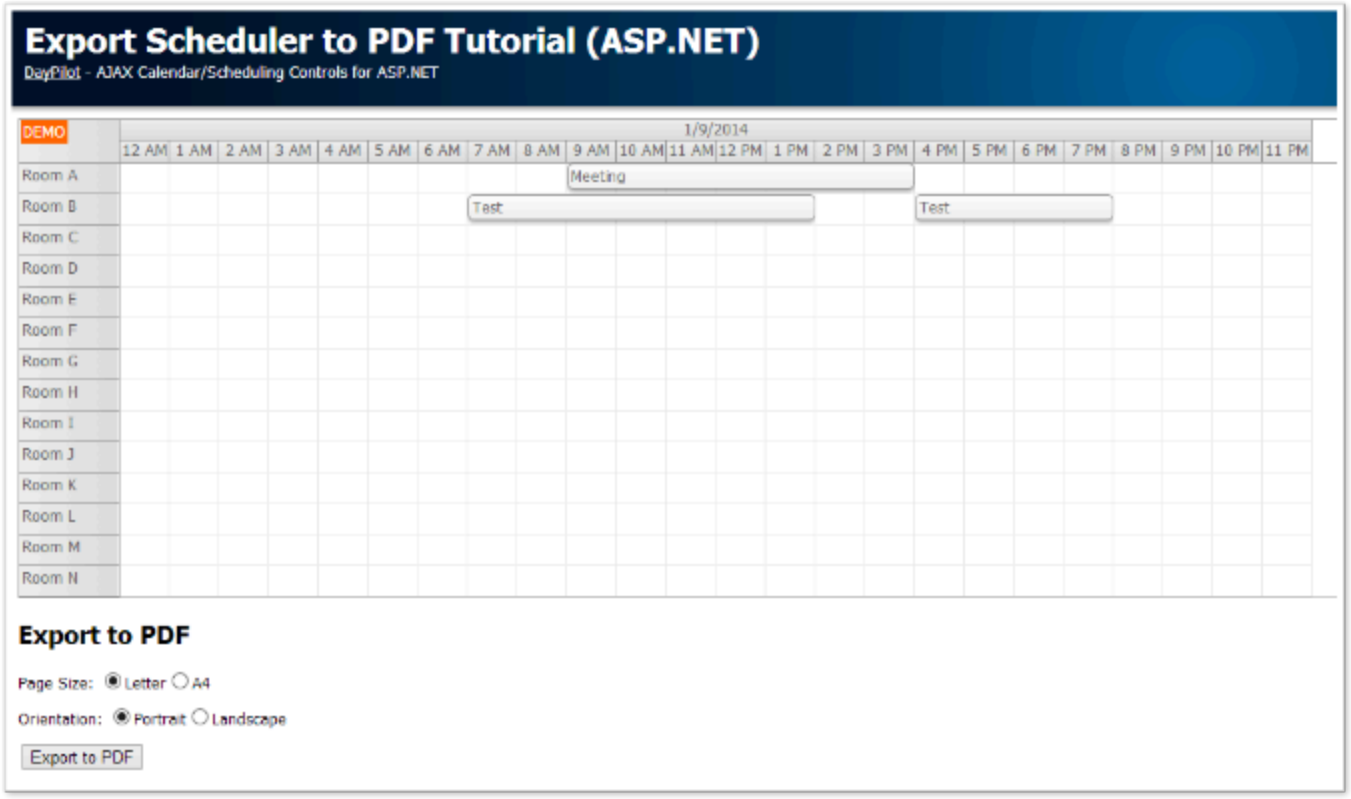
React Scheduler: PDF Export/Printing

How to export the current React Scheduler view to a multi-page PDF document with a specified number of rows per page.



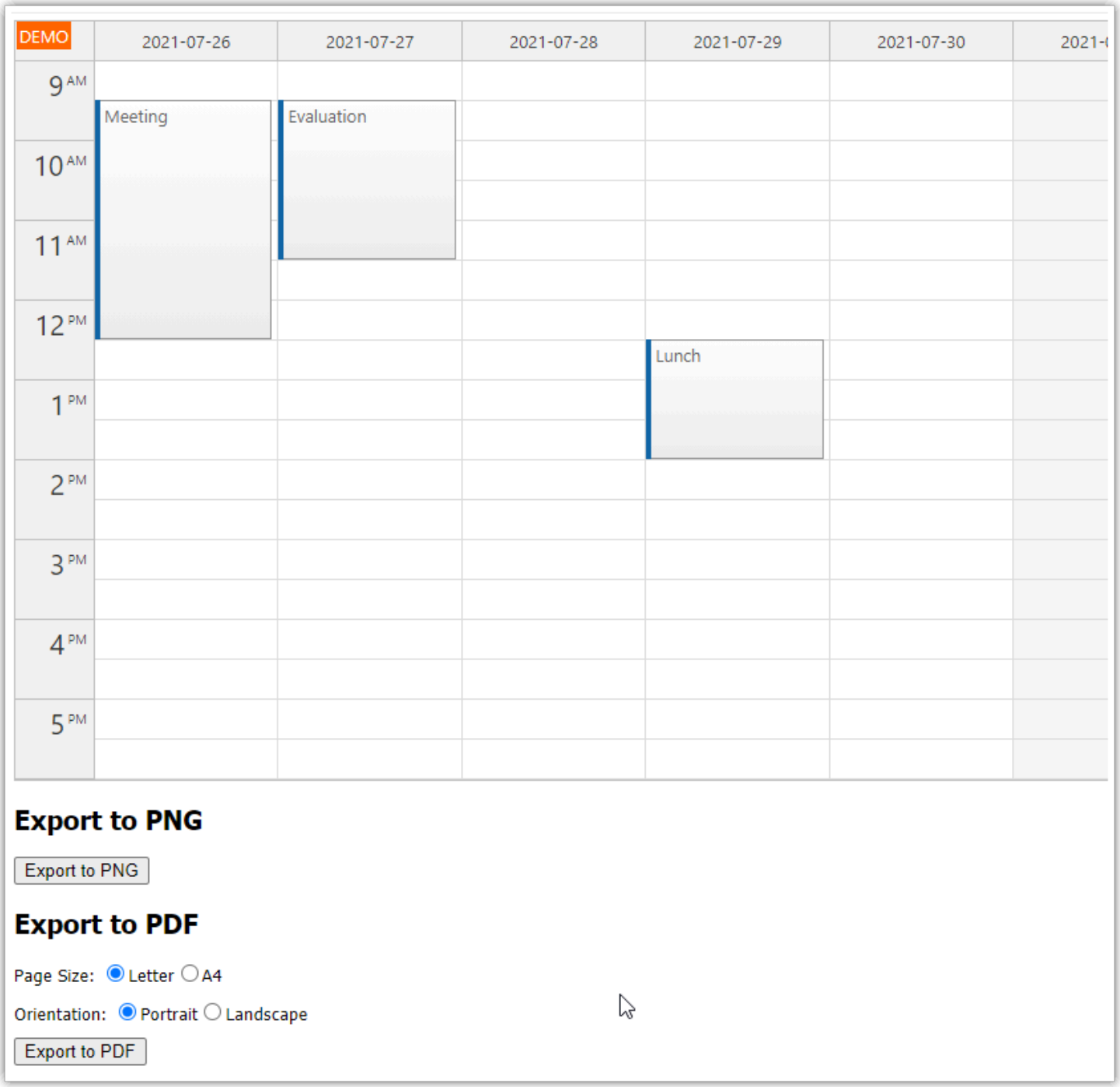
JavaScript Monthly Calendar: PDF Export

How to export the monthly calendar to a PDF document using the built-in image export feature. Adds custom text to the exported PDF file.



ASP.NET Scheduler PDF Export (C#, VB, SQL Server)

This tutorial shows to how export the Scheduler including events/tasks to a PDF document in an ASP.NET application.



ASP.NET Event Calendar PDF Export (C#, VB)

Sample ASP.NET application that exports the weekly calendar with events to PDF. Visual Studio solution with C# and VB source code included.